

Ex 4

```
import java.math.BigInteger;
import java.util.Scanner;
public class RSA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Input two prime numbers
        System.out.print("Enter prime number p: ");
        BigInteger p = sc.nextBigInteger();
        System.out.print("Enter prime number q: ");
        BigInteger q = sc.nextBigInteger();
        // Calculate n and phi
        BigInteger n = p.multiply(q);
        BigInteger phi = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));
        System.out.println("n = " + n);
        System.out.println("phi(n) = " + phi);
        // Input public exponent e
        System.out.print("Enter public key e: ");
        BigInteger e = sc.nextBigInteger();
        // Calculate private key d
        BigInteger d = e.modInverse(phi);
        System.out.println("Public Key = (" + e + ", " + n + ")");
        System.out.println("Private Key = (" + d + ", " + n + ")");
        // Input message
        sc.nextLine();
        System.out.print("Enter message (as a number less than n): ");
        BigInteger message = new BigInteger(sc.nextLine());
        // ----- CONFIDENTIALITY -----
        // Encryption using public key
        BigInteger encrypted = message.modPow(e, n);
        System.out.println("\n--- CONFIDENTIALITY ---");
        System.out.println("Original Message : " + message);
        System.out.println("Encrypted Message: " + encrypted);
        // Decryption using private key
        BigInteger decrypted = encrypted.modPow(d, n);
        System.out.println("Decrypted Message: " + decrypted);

        // ----- AUTHENTICATION -----
        // Digital signature using private key
        BigInteger signature = message.modPow(d, n);
        System.out.println("\n--- AUTHENTICATION ---");
        System.out.println("Digital Signature: " + signature);
        // Verify signature using public key
        BigInteger verifiedMessage = signature.modPow(e, n);

        System.out.println("Verified Message : " + verifiedMessage);
        if (message.equals(verifiedMessage)) {
            System.out.println("Authentication Successful!");
        } else {
            System.out.println("Authentication Failed!");
        }
        sc.close();
    }
}
```

Output

Enter prime number p: 11

Enter prime number q: 13

$n = 143$

$\phi(n) = 120$

Enter public key e: 13

Public Key = (13, 143)

Private Key = (37, 143)

Enter message (as a number less than n): 123

--- CONFIDENTIALITY ---

Original Message : 123

Encrypted Message: 19

Decrypted Message: 123

--- AUTHENTICATION ---

Digital Signature: 84

Verified Message : 123

Authentication Successful!

Ex 5

```
import java.math.BigInteger;
import java.util.Scanner;
public class DiffieHellman {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Public values
        System.out.print("Enter prime number (p): ");
        BigInteger p = sc.nextBigInteger();
        System.out.print("Enter primitive root (g): ");
        BigInteger g = sc.nextBigInteger();
        // Private keys
        System.out.print("Enter private key of Alice: ");
        BigInteger a = sc.nextBigInteger();
        System.out.print("Enter private key of Bob: ");
        BigInteger b = sc.nextBigInteger();
        // Alice calculates public key
        BigInteger A = g.modPow(a, p);
        // Bob calculates public key
        BigInteger B = g.modPow(b, p);
        System.out.println("\n--- Key Exchange ---");
        System.out.println("Alice's Public Key: " + A);
        System.out.println("Bob's Public Key : " + B);
        // Alice calculates shared secret
        BigInteger secretAlice = B.modPow(a, p);
        // Bob calculates shared secret
        BigInteger secretBob = A.modPow(b, p);
        System.out.println("\nAlice's Shared Secret: " + secretAlice);
        System.out.println("Bob's Shared Secret : " + secretBob);
        // Verify
        if (secretAlice.equals(secretBob)) {
            System.out.println("\nSecret Key Exchange Successful!");
            System.out.println("Both Alice and Bob have the same secret key.");
        } else {
            System.out.println("\nSecret Key Exchange Failed!");
        }
        sc.close();
    }
}
```

Output

```
Enter prime number (p): 13
Enter primitive root (g): 2
Enter private key of Alice: 8
Enter private key of Bob: 19
```

```
--- Key Exchange ---
Alice's Public Key: 9
Bob's Public Key : 11
```

```
Alice's Shared Secret: 9
Bob's Shared Secret : 9
```

```
Secret Key Exchange Successful!
Both Alice and Bob have the same secret key.
```